

ACADLYTICS — Project Blueprint

2026-09-22 · @Someone

Academic Performance Intelligence Platform — architecture, plan and 4-member work division

0. Senior Review: Verdict and Changes Made

Verdict: the idea is strong and buildable in 8 weeks — but only if we fix six modelling problems now and cut about a third of the listed features from the MVP. The "Data → Insight → Action → Measurement" loop is the real differentiator; everything else is supporting cast.

Architectural problems in the original proposal (and the fix)

#	Problem	Why it matters	Fix adopted in this blueprint
1	Hierarchy puts Faculty → Section → Assessments	A section (e.g. CSE-A, Sem 3) takes 6 courses from 6 faculty. Assessments belong to a <i>course taught to a section</i> , not to the section	New entity Course Offering = course × section × term × faculty. Assessments, topics analytics, interventions all hang off it
2	No academic term/semester entity	Trends, CT1→CT2 comparisons and reports need a time boundary	Academic Term table; sections are term-scoped
3	Assessments with and without question-wise marks treated the same	Most real faculty sheets only have totals. Topic analytics is impossible without question columns	Every assessment has ≥1 question. A "totals-only" assessment = one question with no topic. Topic analytics shows only where data exists (honest, no fake numbers)
4	One question → one topic	Real questions often span two topics (e.g. "reverse a linked list recursively")	question_topics junction with a weight (default 1.0, split allowed)

#	Problem	Why it matters	Fix adopted in this blueprint
5	Absent = 0 is not addressed	Treating absent as 0 silently destroys averages and falsely flags students	Each mark has a status : present / absent / exempt. Absent excluded from averages, counted against completion
6	"Did the intervention work?" = simple before/after	CT2 may simply be easier than CT1; low scorers also bounce back naturally (regression to the mean). A naive +25 pp claim will be challenged by any faculty reviewer	Report observed change for targeted students <i>and</i> the same change for non-targeted peers in the same offering; show the net difference . Never claim causation in the UI

Scope calls (be critical)

- **Cut from MVP:** attendance module, student login, AI copilot, concept/subtopic analytics, what-if, department-wide report. All are V2/V3.
- **Keep but simplify:** grading. Do not hard-code SRM grading; store a configurable grade-band scheme per course offering.
- **Keep, it's cheap and impressive:** item difficulty + discrimination index (upper/lower 27% method). It is a subtraction, fully explainable, and faculty recognise it.
- **Unnecessary:** Celery/Redis job queues, microservices, Kubernetes, text-to-SQL for AI, ML "failure prediction". A section is ~60–70 students; a CT import is ~200 rows. FastAPI + PostgreSQL handles this synchronously.
- **Explicitly avoid:** any ML risk score. Rule-based, explainable flags only. A black-box "will fail" label is an ethical and demo liability.

Assumptions made (confirm with guide)

- Scale: 1 department, ~4 batches, ~12 sections, ~800 students, ~40 faculty.
- Pass mark and grade bands are configured per course; default pass = 50%.
- Excel upload is per assessment, one sheet, one row per student.

1. Executive Summary

ACADLYTICS is an Academic Performance Intelligence Platform that turns raw assessment marks into explainable, actionable insight for faculty and department heads.

- **What it is:** a web platform where faculty upload or enter marks (CTs, FTs, quizzes, assignments, labs) at question level, and the system automatically computes statistics, topic mastery, student trends and academic-attention flags.
- **Problem solved:** faculty currently spend hours per assessment cycle in spreadsheets, and still cannot easily see *which topics* a section is weak in, *which students* are declining, or *whether* a remedial session helped.
- **Users:** HOD/Admin (department oversight), Faculty (daily use), Students (optional, read-only, V2).
- **Why useful:** one upload replaces manual averaging, charting and shortlisting; every number is traceable to the marks behind it.
- **What makes it different:** it closes the loop. A marks system stores data; ACADLYTICS answers *what happened, why, who is affected, what to do, and whether it worked* — through question-to-topic mapping, rule-based attention flags with visible reasons, and intervention outcome tracking against a peer baseline.

Design principle: deterministic analytics first; AI only narrates verified numbers, never produces them.

2. Problem Statement

Faculty in a CSE department manage academic data for 2–4 sections per course in disconnected spreadsheets, and the analysis that should drive teaching decisions is either done late, done manually, or not done at all.

Current workflow

1. Faculty conducts CT-1, evaluates scripts question by question.
2. Marks go into a personal Excel file per section (layouts differ per faculty).
3. Faculty computes totals, averages, pass % manually or with ad-hoc formulas.
4. Marks are re-typed or copied into the institutional portal.
5. Section comparison and HOD reports are compiled by hand at the end of term.

Problems with spreadsheets

- No single source of truth; files are duplicated, emailed and overwritten.
- No validation: wrong register numbers, marks above maximum and duplicate rows go unnoticed.
- Absent vs zero is inconsistent across files.
- No access control or audit trail on sensitive student data.

Problems with manual analysis

- Each new assessment means redoing formulas and charts.
- Cross-section and cross-assessment comparison requires merging files.
- Analysis happens after the teaching window where it could have helped.

Specific gaps

- **Weak topics:** question-wise marks exist on paper but are rarely mapped to syllabus topics, so faculty cannot say "Section B is weak in Recursion" with evidence.
- **Student trends:** a student sliding from 78% to 52% across three assessments is invisible when each assessment is a separate sheet.
- **Interventions:** remedial sessions happen, but nobody records who attended or compares performance afterwards, so the department cannot learn what works.

3. Proposed Solution

ACADLYTICS replaces the spreadsheet workflow with a validated pipeline that ends in a measurable action.

```
flowchart LR
    A[Data<br/>Excel / manual entry] --> B[Processing<br/>validate, match, store]
    B --> C[Analytics<br/>stats, topics, trends]
    C --> D[Insights<br/>weak topics, attention flags]
    D --> E[Intervention<br/>faculty action on a student group]
    E --> F[Measurement<br/>net change vs peers]
    F -.next assessment.-> A
```

The loop restarts with every assessment, so each intervention is measured on real follow-up data.

Stage	What the system does	Example output
Data	Accepts a CT1.xlsx or grid entry against a defined assessment + questions	186 rows received
Processing	Matches register numbers, validates marks, previews errors, imports on confirm	182 matched, 2 unknown, 1 duplicate, 1 over max
Analytics	Computes assessment, question, topic, student and section statistics	Q4 avg 37%; Recursion 48%

Stage	What the system does	Example output
Insights	Applies configured rules; explains each flag	23 students below 50% in Recursion
Intervention	Faculty creates an action for a student group and topic	"Recursion remedial, 17 students, 24 Sep"
Measurement	Compares topic scores before/after for target group and non-target peers	Target +25 pp, peers +6 pp, net +19 pp

4. Objectives

Each objective is testable at the final demo.

ID	Objective	Measure of success
O1	Centralise academic records for a department	All MVP entities stored in PostgreSQL; zero mock data after Week 4
O2	Reduce assessment processing time	Upload → validated → dashboard updated in < 2 min for a 70-student section (vs. ~1–2 h manual)
O3	Guarantee data quality at import	100% of the 8 defined validation rules caught in the import test suite
O4	Provide topic-level insight	Topic averages available for any assessment with question-topic mapping
O5	Explainable attention flags	Every flag shows rule, threshold and actual value; 0 unexplained flags
O6	Measure interventions	Before/after + peer-baseline comparison computed for every intervention with follow-up data
O7	Secure access	JWT auth, RBAC on every endpoint, faculty see only their offerings (tested)
O8	Correct analytics	Analytics unit tests match hand-computed spreadsheet values to 2 decimal places
O9	Professional reporting	Section and assessment reports exportable as PDF, Excel and CSV

5. Target Users

Three roles; authorization is always scoped to the offerings a user is linked to.

Capability	Admin / HOD	Faculty	Student (V2, optional)
Manage users, departments, terms, courses, sections	Yes	No	No
Assign faculty to course offerings	Yes	No	No
Manage students (add, edit, deactivate, CSV import)	Yes	Own sections only (edit)	No
Create assessments, questions, topic mapping	View all	Own offerings	No
Enter / import marks	No (view)	Own offerings	No
View analytics & dashboards	Whole department	Own offerings	Own marks and topic profile only
Configure thresholds & grading	Department defaults	Per-offering override	No
Create interventions	View all	Own offerings	No
Reports	Department, section, any	Own offerings	Own report card
AI copilot (V3)	Department scope	Own offerings	No

Recommendation on Student role: build it last and read-only. Never show a student their rank or other students' data. If time is short, drop it entirely; the faculty loop is the product.

6. Complete Feature List (MVP / V2 / V3)

Rule for the team: nothing in V2 starts until every MVP item passes its Definition of Done. Nothing in V3 starts before Week 7.

MVP — must ship (Weeks 1-6)

- Auth: login, logout, JWT (access + refresh), bcrypt/argon2 hashing, RBAC, protected routes.
- Org structure: department, academic term, course, section, course offering (faculty assignment).
- Students: CRUD, soft-deactivate, search, filter, CSV/Excel bulk import, profile page.
- Assessments: CRUD with type, date, max marks, weightage; questions with max marks.
- Topics: dynamic per course, question → topic mapping (weighted).
- Marks: spreadsheet-style grid entry/edit, status (present/absent/exempt).
- Smart Excel import: template download, parse, validate, preview, fix/exclude rows, confirm, import.
- Core analytics: totals, %, mean, median, std dev, min/max, pass %, grade distribution, weighted score.
- Question analytics: average %, difficulty band.
- Topic analytics: topic average per assessment and per offering; weak-student counts.
- Student profile: history, topic strengths/weaknesses, trend label.
- Trend classification (improving / stable / declining) with visible rule.
- Academic attention flags (rule-based, explainable, configurable thresholds).
- Faculty dashboard + section comparison.
- Interventions: create, target students by topic filter, measure before/after vs peers.
- Reports: section and assessment report, PDF + Excel + CSV.

V2 — advanced (Week 7, only if MVP is green)

- Item discrimination index (upper/lower 27%) and "questions to review" list.
- What-if simulator (projected section average) — cheap, deterministic, good demo.
- Student role (read-only own profile).
- Topic report and department (HOD) report.
- Audit log viewer; import history with rollback of a batch.
- Email notification to faculty when flags change (optional).

V3 — AI and future (Week 7-8 stretch, or future scope)

- Academic Copilot: natural-language questions answered by calling fixed analytics tools.
- AI-generated section summary / HOD narrative from verified numbers.

- Concept/subtopic hierarchy analytics (schema supports it from day one via `topic.parent_id`).
- Attendance import and attendance-based flags.
- Integration with institutional ERP/portal.
- Multi-department / multi-college tenancy.

Explicitly rejected

- ML "failure prediction" model: too little data per college (a few hundred students), not explainable, ethically risky.
- Text-to-SQL chatbot: unsafe and unreliable; replaced by tool-calling over fixed endpoints.
- Real-time websockets, microservices, job queues: no requirement justifies them.

7. Functional Requirements

IDs are referenced by tasks and tests. Priority: MVP unless marked.

ID	Module	Requirement
FR-01	Auth	Users log in with email + password and receive an access token (15 min) and refresh token (7 days)
FR-02	Auth	Passwords are stored as argon2/bcrypt hashes; plain text never logged
FR-03	Auth	Every API route enforces role and offering-scope checks server-side
FR-04	Auth	Admin can create, deactivate and reset users
FR-05	Org	Admin manages departments, terms, courses, sections and course offerings
FR-06	Students	Create, edit, soft-deactivate, search (name/reg no), filter (batch, semester, section)
FR-07	Students	Bulk import students from CSV/XLSX with the same validate-preview-confirm flow as marks
FR-08	Assessments	Faculty creates assessments on an offering: name, type, date, max marks, weightage, sequence
FR-09	Assessments	System warns when total weightage of an offering exceeds 100

ID	Module	Requirement
FR-10	Questions	Faculty defines questions (label, max marks); sum of question max must equal assessment max
FR-11	Topics	Topics are defined per course (optionally nested); faculty maps each question to 0..n topics with weights
FR-12	Marks	Faculty enters/edits marks in a grid; each cell has score + status (present/absent/exempt)
FR-13	Marks	Marks above question max or below 0 are rejected
FR-14	Import	Faculty downloads a generated template for an assessment (reg no, name, Q columns)
FR-15	Import	Upload is parsed, validated and previewed without writing marks
FR-16	Import	Preview shows counts and row-level errors; faculty may fix values inline or exclude rows
FR-17	Import	Confirm writes marks transactionally (all or nothing) and records an import batch
FR-18	Analytics	Assessment stats: mean, median, std dev, min, max, pass %, grade distribution, completion %
FR-19	Analytics	Question stats: average %, difficulty band, score distribution
FR-20	Analytics	Topic stats per assessment and cumulative per offering; count of students below threshold
FR-21	Analytics	Student weighted score, topic profile, trend label with the rule shown
FR-22	Analytics	Section comparison across offerings of the same course and term
FR-23	Attention	Rules evaluated after each import/edit; each flag stores rule, threshold, actual value
FR-24	Attention	Thresholds configurable at department level, overridable per offering
FR-25	Intervention	Faculty creates an intervention: topic, target students, action, resources, date, follow-up assessment

ID	Module	Requirement
FR-26	Intervention	System computes pre/post topic score for targets and non-target peers, and net change
FR-27	Reports	Section and assessment reports export to PDF, XLSX, CSV
FR-28	Dashboard	Faculty dashboard shows KPIs, trends, weak topics and attention list for selected offering
FR-29	Audit	Marks changes and imports record who, when, old/new value (V2 viewer)
FR-30	V2	Discrimination index per question; what-if projection
FR-31	V3	Copilot answers questions only through whitelisted analytics tools, citing the numbers used

8. Non-Functional Requirements

Targets are sized for one department (~800 students) on a single small cloud VM.

Area	Requirement
Security	HTTPS only; JWT signed HS256 with secret from env; refresh-token rotation; RBAC + offering scope on every query; SQLAlchemy ORM only (no string SQL); upload size limit 5 MB, xlsx/csv only; rate limit on /auth/login (5/min/IP)
Performance	p95 API < 300 ms for dashboard reads; import of 200 rows × 10 questions validated in < 3 s; PDF report < 5 s
Scalability	Stateless API containers; analytics computed via SQL aggregates + per-offering cache invalidated on write; schema supports multiple departments
Availability	Target 99% during demo/eval period; health endpoint; Docker restart policy; nightly pg_dump backup

Area	Requirement
Maintainability	Layered backend (router → service → repository); typed contracts (Pydantic ↔ generated TS types); ≥70% coverage on analytics and import services; lint + format in CI
Usability	Any core task ≤ 3 clicks from dashboard; import wizard understandable without training; responsive down to 1280 px (laptop) and usable on tablet
Data privacy	Student PII visible only to authorised roles; no PII sent to LLM beyond what a query needs (prefer reg-no pseudonyms); follows spirit of India's DPDP Act 2023: purpose limitation, access control, deletion on request
Reliability	Imports are atomic; analytics are deterministic and reproducible from stored marks; soft deletes only for students and assessments
Explainability	Every derived label (grade, trend, flag, difficulty) shows the formula/threshold behind it via tooltip or detail panel

9. System Architecture

A modular monolith: one React SPA, one FastAPI service split into domain modules, one PostgreSQL database. This is the right size for 4 developers and 8 weeks.

9.1 Layered view

```

flowchart TD
    UI[React + TS SPA<br/>Tailwind, Recharts, TanStack Query] --> API[FastAPI<br/>routers<br/>/api/v1]
    API --> AUTH[Auth dependency<br/>JWT + RBAC + scope]
    AUTH --> SVC[Service layer<br/>business rules]
    SVC --> ANA[Analytics engine<br/>pure functions]
    SVC --> REPO[Repository layer<br/>SQLAlchemy 2.0]
    ANA --> REPO
    REPO --> DB[(PostgreSQL 16)]
  
```

Routers never touch the database directly; analytics functions take plain data in and return plain data out, so they are unit-testable without a database.

9.2 Import pipeline view

```

flowchart LR
  X[Excel / CSV] --> P[Parser<br/>pandas + openpyxl]
  P --> V[Validator<br/>8 rules]
  V --> S[Staging<br/>import_batches]
  S --> PR[Preview UI]
  PR -->|confirm| W[Transactional write<br/>marks]
  W --> R[Recompute<br/>summaries + flags]

```

9.3 AI view (V3)

```

flowchart LR
  DB[(PostgreSQL)] --> AE[Analytics endpoints]
  AE --> T[Copilot tools<br/>whitelisted]
  T --> L[LLM<br/>narrates only]
  L --> V[Number checker]
  V --> U[Answer + sources]

```

9.4 Backend module layout

Module	Owns
core	config, security (JWT, hashing), DB session, error handling, pagination
auth / users	login, refresh, user admin
org	departments, terms, courses, sections, offerings
students	student CRUD, student import
assessments	assessments, questions, topics, question_topics
marks	mark grid read/write
imports	parse, validate, stage, confirm
analytics	stats functions, summary cache, trend, attention rules
interventions	interventions + outcome computation
reports	PDF (WeasyPrint), XLSX (openpyxl), CSV
copilot (V3)	tool registry, LLM client, verification

9.5 Key technical decisions

- **Compute strategy:** analytics are computed on write, not on every read. After an import or mark edit, the service recomputes `student_assessment_summary` and `student_topic_summary` rows for that assessment, then re-evaluates attention flags for that offering. Dashboards read these small tables. At our scale this takes well under a second, so no job queue.
- **Contracts:** FastAPI generates OpenAPI; frontend generates TypeScript types with `openapi-typescript`. One source of truth.
- **Migrations:** Alembic, owned by one person (M1), reviewed by all.
- **Frontend state:** TanStack Query for server state; no Redux.
- **PDF:** HTML templates (Jinja2) → WeasyPrint, so report layout is just HTML/CSS.
- **Deployment:** docker-compose (nginx + api + postgres) on one VM (AWS EC2 t3.small / Azure B1ms). Optional: managed Postgres (RDS / Azure Database).

10. Database Design

22 tables in PostgreSQL (16 domain + 2 analytics caches + 4 support). All tables have `id` (UUID or BIGSERIAL PK), `created_at`, `updated_at`. Soft-deletable tables have `is_active`.

10.1 Tables

Table	Purpose	Key columns	Foreign keys / constraints
departments	Top of hierarchy	name, code	UNIQUE(code)
users	Admin/HOD, faculty, student logins	email, password_hash, full_name, role (enum: admin, faculty, student), is_active	department_id → departments; student_id → students (nullable, V2)
refresh_tokens	Token rotation / logout	token_hash, expires_at, revoked_at	user_id → users
academic_terms	Semester window	name ("Odd 2026-27"), start_date, end_date, is_current	—
courses	Subject catalogue	code ("21CSC201J"), name, credits	department_id → departments; UNIQUE(department_id, code)
sections	Student cohort for a term	name ("A"), batch ("2025-29"), semester	department_id, term_id; UNIQUE(term_id, batch, name)
students	Student master	register_no, name, email, batch, is_active	department_id, section_id → sections; UNIQUE(register_no)
course_offerings	Course taught to a section in a term by a faculty	pass_percent (default 50), config JSONB (threshold overrides)	course_id, section_id, term_id, faculty_id → users; UNIQUE(course_id, section_id, term_id)

Table	Purpose	Key columns	Foreign keys / constraints
grading_schemes	Grade bands	name, bands JSONB ([{grade:"O", min:91}, ...])	department_id; course_offerings.grading_scheme_id → here
topics	Dynamic syllabus topics	name, description, sort_order	course_id → courses; parent_id → topics (nullable, subtopics); UNIQUE(course_id, parent_id, name)
assessments	CT/FT/quiz/etc.	name, type (enum), date, max_marks, weightage, sequence_no, is_published	offering_id → course_offerings
questions	Items within an assessment	label ("Q1a"), max_marks, order_no	assessment_id → assessments; UNIQUE(assessment_id, label)
question_topics	Question ↔ topic mapping	weight (0-1, default 1)	question_id, topic_id; PK(question_id, topic_id)
marks	Atomic fact	score NUMERIC(5,2) nullable, status (present/absent/exempt), updated_by	student_id, question_id; UNIQUE(student_id, question_id); CHECK(score ≥ 0)
student_assessment_summary	Cache: per student per assessment	total, percentage, grade, is_pass, status, computed_at	student_id, assessment_id; PK(student_id, assessment_id)
student_topic_summary	Cache: per student per topic per assessment	earned, possible, percentage	student_id, topic_id, assessment_id; PK(all three)
attention_flags	Explainable flags	rule_code, severity, threshold, actual_value, message, status (open/acknowledged/resolved), computed_at	student_id, offering_id; topic_id (nullable)
interventions	Faculty action	title, action_type, description, resources, conducted_on, status	offering_id, topic_id, created_by → users; baseline_assessment_id, followup_assessment_id → assessments
intervention_students	Target group	attended (bool)	intervention_id, student_id; PK(both)
import_batches	Import staging + history	kind (marks/students), file_name, status (previewed/committed/discarded), summary JSONB, rows JSONB, committed_at	assessment_id (nullable), uploaded_by → users
audit_logs	Change history	entity, entity_id, action, old_value JSONB, new_value JSONB	actor_id → users
settings	Department thresholds	key, value JSONB	department_id

10.2 Textual ER diagram

```
erDiagram
    DEPARTMENT ||--o{ COURSE : offers
    DEPARTMENT ||--o{ SECTION : has
    TERM ||--o{ SECTION : scopes
    SECTION ||--o{ STUDENT : contains
    COURSE ||--o{ TOPIC : defines
    COURSE ||--o{ OFFERING : taught_as
    SECTION ||--o{ OFFERING : takes
    USER ||--o{ OFFERING : teaches
    OFFERING ||--o{ ASSESSMENT : has
    ASSESSMENT ||--o{ QUESTION : contains
    QUESTION }o--o{ TOPIC : maps_via_question_topics
    STUDENT ||--o{ MARK : scores
    QUESTION ||--o{ MARK : receives
    OFFERING ||--o{ INTERVENTION : runs
    INTERVENTION }o--o{ STUDENT : targets
    STUDENT ||--o{ ATTENTION_FLAG : raises
```

10.3 Design notes

- `marks` is the only source of truth. Both summary tables can be dropped and rebuilt from it at any time (`POST /admin/recompute`).
- Indexes: `marks(question_id)`, `students(section_id)`, `assessments(offering_id, sequence_no)`, `student_topic_summary(topic_id, assessment_id)`, trigram index on `students.name` for search.
- Enums as PostgreSQL enums or CHECK constraints; keep them in one `enums.py` shared with the frontend via OpenAPI.
- Students changing section next semester: new section row per term, `students.section_id` updated; historical data is safe because marks link to questions → assessments → offerings, not to the student's current section.

11. API Architecture

REST, JSON, base path `/api/v1`, OpenAPI at `/docs`. All endpoints require a Bearer token except login/refresh. "Scope" = faculty may only touch offerings they teach; admin may touch all.

11.1 Conventions

- Lists: `?page=1&size=25&sort=name&q=search` → `{items: [], total, page, size}`.

- Errors: `{error: {code: "MARK_EXCEEDS_MAX", message, details}}` with correct HTTP status (400, 401, 403, 404, 409, 422).
- IDs in paths; filters in query strings; no verbs in paths except explicit actions (`/confirm` , `/recompute` , `/export`).

11.2 Endpoints

Group	Method + path	Purpose	Auth
Auth	POST <code>/auth/login</code>	<code>{email, password}</code> → <code>{access_token, refresh_token, user}</code>	Public
Auth	POST <code>/auth/refresh</code>	Rotate refresh token	Refresh token
Auth	POST <code>/auth/logout</code>	Revoke refresh token	Any
Auth	GET <code>/auth/me</code>	Current user + role + offerings	Any
Users	GET/POST <code>/users</code> , PATCH/DELETE <code>/users/{id}</code>	Manage users (DELETE = deactivate)	Admin
Org	GET/POST <code>/departments</code> , <code>/terms</code> , <code>/courses</code> , <code>/sections</code>	Master data	Read: any; write: admin
Org	GET/POST <code>/offerings</code> , PATCH <code>/offerings/{id}</code>	Assign faculty; configure pass %, grading, thresholds	Write: admin (config: owner faculty)
Students	GET <code>/students?section_id&batch&q</code>	Search/filter	Admin; faculty (scoped)
Students	POST <code>/students</code> , PATCH <code>/students/{id}</code> , POST <code>/students/{id}/deactivate</code>	CRUD	Admin
Students	GET <code>/students/{id}</code>	Profile header	Scoped
Assessments	GET <code>/offerings/{id}/assessments</code> , POST same	List/create	Scoped faculty
Assessments	GET/PATCH/DELETE <code>/assessments/{id}</code>	Detail incl. questions	Scoped faculty
Questions	PUT <code>/assessments/{id}/questions</code>	Replace full question list <code>[{label, max_marks, topics:[{topic_id, weight}]]</code> ; validates sum = max	Scoped faculty

Group	Method + path	Purpose	Auth
Topics	GET/POST /courses/{id}/topics, PATCH/DELETE /topics/{id}	Dynamic topic tree	Faculty of that course; admin
Marks	GET /assessments/{id}/marks	Grid: {questions[], rows: [{student, cells: [{question_id, score, status}]]}]}	Scoped
Marks	PUT /assessments/{id}/marks	Bulk upsert changed cells; triggers recompute	Scoped faculty
Import	GET /assessments/{id}/import/template	Download XLSX template	Scoped faculty
Import	POST /assessments/{id}/import (multipart)	Parse + validate → {batch_id, summary, rows[], errors[]} ; nothing written to marks	Scoped faculty
Import	PATCH /imports/{batch_id}/rows	Fix values / exclude rows; re-validate	Owner
Import	POST /imports/{batch_id}/confirm	Transactional commit + recompute → {imported, skipped}	Owner
Import	POST /students/import (+ /confirm)	Same flow for students	Admin
Analytics	GET /analytics/dashboard?offering_id=	KPIs, trend series, weak topics, attention count	Scoped
Analytics	GET /analytics/assessments/{id}	Stats, grade distribution, histogram	Scoped
Analytics	GET /analytics/assessments/{id}/questions	Per-question avg, difficulty (V2: discrimination)	Scoped
Analytics	GET /analytics/offerings/{id}/topics	Topic averages + below-threshold counts, per assessment and cumulative	Scoped
Analytics	GET /analytics/students/{id}? offering_id=	History, topic profile, trend + rule, flags, generated insight sentence	Scoped
Analytics	GET /analytics/sections/compare? course_id&term_id	Section comparison	Admin; faculty of that course
Analytics	GET /analytics/offerings/{id}/attention	Flag list with factors	Scoped

Group	Method + path	Purpose	Auth
Analytics	POST /analytics/offerings/{id}/what-if (V2)	{student_ids, topic_id, delta_pp} → current vs projected	Scoped
Interventions	GET/POST /offerings/{id}/interventions	List/create {title, topic_id, student_ids[], action_type, resources, conducted_on, baseline_assessment_id, followup_assessment_id}	Scoped faculty
Interventions	GET/PATCH /interventions/{id}	Detail + outcome {target: {pre, post, n}, peers: {pre, post, n}, net_change}	Scoped
Reports	GET /reports/{type}/{id}?format=pdf xlsx csv	type ∈ section, assessment, student, topic (V2), department (V2)	Scoped
AI (V3)	POST /copilot/ask	{question, offering_id?} → {answer, tool_calls[], sources[]}	Faculty, admin
Admin	POST /admin/recompute?offering_id=	Rebuild caches and flags	Admin
System	GET /health	Liveness + DB check	Public

12. Analytics Engine

All analytics are deterministic Python functions over marks, with thresholds read from config. Every output carries the inputs needed to explain it.

12.1 Handling missing data (applies to every formula)

- `present` → included. `absent` → excluded from averages, counted as not completed. `exempt` → excluded from everything.
- A student is "assessed" on an assessment only if at least one question is `present`.
- Any statistic shows its **n** (students included). If $n < 5$, the UI shows "insufficient data" instead of a label.

12.2 Student-level

Assessment total and percentage for student s on assessment a :

$$T_{\{s,a\}} = \sum_{q \in a} x_{\{s,q\}}, \quad P_{\{s,a\}} = 100 \cdot \frac{T_{\{s,a\}}}{M_a}$$

where x is the score on question q and M is the assessment's max marks. Grade = first band in the offering's grading scheme with $P \geq \text{band.min}$. Pass = $P \geq \text{offering.pass_percent}$.

Weighted course score over completed assessments (renormalised so missing future assessments don't drag it down):

$$W_s = \frac{\sum_{a \in A_s} P_{\{s,a\}} \cdot w_a}{\sum_{a \in A_s} w_a}$$

Topic score for student s on topic t (across the chosen assessment set, weight = question_topics.weight):

$$TS_{\{s,t\}} = 100 \cdot \frac{\sum_{q \mapsto t} x_{\{s,q\}} \cdot \omega_{\{q,t\}}}{\sum_{q \mapsto t} m_q \cdot \omega_{\{q,t\}}}$$

12.3 Group-level (assessment, section, topic)

Metric	Definition
Mean	Average of P over assessed students
Median	50th percentile of P (robust to outliers; show next to mean)
Std dev	Population std dev of P
Highest / lowest	max / min of P , with student reference
Pass %	$100 \times (\text{students with } P \geq \text{pass_percent}) / \text{assessed students}$
Completion %	$100 \times \text{assessed} / \text{enrolled active students}$
Grade distribution	Count per grade band
Histogram	Counts in 10-point bins (0–9, 10–19, ..., 90–100)
Question average	Mean of $100 \times x/m$ over present students
Topic average	Same formula as TS but summed over all students in the group (pooled), not mean of means
Weak students in topic	Count of students with $TS < \text{topic_threshold}$ (default 50)

Metric	Definition
Section comparison	Mean, median, pass % per offering of same course + term, plus per-topic means

12.4 Question difficulty and discrimination

Difficulty index p = mean of x/m over present students (0 to 1). Bands: $p \geq 0.80$ Easy, 0.50–0.79 Moderate, 0.30–0.49 Difficult, < 0.30 Very difficult.

Discrimination (V2) uses the classical upper–lower 27% method: rank students by assessment total, take top 27% (U) and bottom 27% (L).

$$D_q = \bar{p}_{q,U} - \bar{p}_{q,L}$$

Interpretation: $D \geq 0.40$ very good, 0.30–0.39 good, 0.20–0.29 marginal, < 0.20 review question, negative = likely flawed (strong students did worse). Only shown when $n \geq 20$. This is explainable in one sentence, which is why it's included; point-biserial correlation is not needed.

12.5 Trend classification

Uses the student's P series over assessments in the offering, ordered by sequence_no.

```
if completed_assessments < 2: label = "Insufficient data"
elif completed_assessments == 2: slope = P[-1] - P[-2]
else: slope = least_squares_slope(x = 1..k, y = P)    # pp per assessment

if slope >= +TREND_DELTA (default 5): "Improving"
elif slope <= -TREND_DELTA: "Declining"
else: "Stable"
```

The UI shows: "Declining: scores fell by 7.5 percentage points per assessment across CT1, CT2, FT1 (threshold 5)."

Caveat to state in the report: comparing raw % across assessments ignores difficulty differences. Optional refinement (V2): use each student's z-score within the assessment instead of raw %, so a hard CT2 does not make everyone "decline".

12.6 Decline detection

```
prev_mean = mean(P over all earlier assessments)
drop = P_latest - prev_mean
sharp_decline = drop <= -DECLINE_DROP (default 15)
```

12.7 Academic attention rules

Rules are data (`settings`), not code branches. Each fired rule becomes a factor on the flag.

Code	Rule	Default threshold	Severity
R1_LOW_OVERALL	$W_s < \text{threshold}$	50%	High
R2_LOW_TOPIC	$TS_{s,t} < \text{threshold}$ for any topic with ≥ 10 possible marks	40%	Medium (one flag per topic)
R3_SHARP_DECLINE	$\text{drop} \leq -\text{threshold}$	15 pp	Medium
R4_DECLINING_TREND	$\text{trend} = \text{Declining}$	5 pp/assessment	Low
R5_LOW_COMPLETION	$\text{completion} < \text{threshold}$	75%	Medium
R6_FAILED_LATEST	$P_{\text{latest}} < \text{pass_percent}$	offering pass %	Medium

A student is listed as **Requiring academic attention** if any High rule fires or ≥ 2 Medium rules fire. Display text: "Flagged for additional academic support based on: overall 44% (threshold 50%); Recursion 31% (threshold 40%)." Never "will fail".

12.8 Intervention outcome

For intervention on topic t with target group G , peers N (other active students in the offering), baseline assessment set B and follow-up assessment F :

$$\Delta_G = \overline{TS}_{G,t}(F) - \overline{TS}_{G,t}(B), \quad \Delta_N = \overline{TS}_{N,t}(F) - \overline{TS}_{N,t}(B), \quad \text{Net} = \Delta_G - \Delta_N$$

Shown as: "Targeted students improved 42% $\rightarrow$ 67% (+25 pp); other students changed +6 pp; net +19 pp. Observational — not a controlled experiment." Only students with data in both B and F are counted; n is shown. Requires the follow-up assessment to contain at least one question mapped to t .

12.9 Generated insight sentence (no AI)

Template-based: strong topics = $TS \geq 75$, weak = $TS < 50$. "{Name} shows strong performance in {strong} but needs practice in {weak}." Deterministic and testable.

12.10 What-if (V2)

Apply $+\delta$ pp to TS of chosen students on topic t (capped at 100), convert back to question scores proportionally, recompute section mean. Return current, projected, and difference.

13. Excel Import Architecture

The import never writes marks until the faculty confirms, and the confirm is a single database transaction.

```
stateDiagram-v2
    [*] --> Uploaded
    Uploaded --> Parsed: read sheet
    Parsed --> Rejected: header/format error
    Parsed --> Previewed: row validation
    Previewed --> Previewed: fix or exclude rows
    Previewed --> Committed: confirm (0 blocking errors)
    Previewed --> Discarded: cancel / 24h expiry
    Committed --> [*]
```

13.1 Steps

1. **Template (recommended path):** faculty downloads a template generated from the assessment — columns `Register No | Name | Q1 | Q2 | ...`, row 2 shows max marks, data validation on cells. Most errors never happen.
2. **Upload:** multipart, ≤ 5 MB, `.xlsx / .csv` only, MIME + extension check.
3. **Parse:** pandas `read_excel(engine="openpyxl", dtype=str)`; strip whitespace; normalise headers (case-insensitive, "Q1", "Q 1", "q1" $\rightarrow$ Q1); detect header row within first 5 rows.
4. **Column mapping:** auto-map Q-columns to question labels. Unmapped columns $\rightarrow$ faculty maps them in the UI (dropdown). Extra columns ignored with a warning.
5. **Validate** each row (below) $\rightarrow$ staged in `import_batches.rows` JSONB with per-cell status.
6. **Preview:** summary counters + table with red/amber cells; filter "errors only".
7. **Fix / exclude:** inline edit a cell or tick "exclude row"; revalidate that row server-side.
8. **Confirm:** allowed only when blocking errors = 0 (after fixes/exclusions). Upsert marks in one transaction; write audit rows; mark batch committed.
9. **Recompute:** summaries for the assessment, topic summaries, attention flags for the offering. Return counts. Dashboard refetches.

13.2 Validation rules

#	Rule	Level	Example message
V1	Required headers present (register no + ≥ 1 question column)	File-blocking	"Column 'Register No' not found"
V2	Register no exists in offering's section	Row-blocking	"RA2511003010123 not in CSE-A"
V3	Duplicate register no in file	Row-blocking (both rows)	"Duplicate of row 14"
V4	Score is numeric	Cell-blocking	"'twelve' is not a number"
V5	$0 \leq \text{score} \leq \text{question max}$	Cell-blocking	"Q3 = 7, max is 5"
V6	Blank cell	Warning $\rightarrow$ imported as absent (faculty can override to 0)	"Q2 blank — will be marked absent"
V7	"AB", "A", "-"	Info $\rightarrow$ status absent	—
V8	Name mismatch with DB (fuzzy ratio < 80)	Warning	"Name 'Ravi K' vs 'Ravikumar S'"
V9	Student active in section but missing from file	Warning	"3 students not in file"
V10	Marks already exist for this assessment	Warning $\rightarrow$ confirm overwrites; old values kept in audit log	"62 existing marks will be updated"

13.3 Summary returned to UI

```
{total_rows: 186, matched: 182, not_found: 2, duplicates: 1, missing_marks: 3, exceeds_max: 1, missing_students: 3, blocking_errors: 4}
```

13.4 Student import

Same engine, different schema: register no format regex, email format, section exists, duplicate against DB $\rightarrow$ "update existing?" choice.

14. AI Copilot Architecture (V3)

The LLM never sees raw tables and never computes a number; it chooses from a fixed set of analytics tools and writes prose around their outputs.

```
sequenceDiagram
    participant F as Faculty
    participant C as Copilot service
    participant L as LLM
    participant A as Analytics API
    F->>C: Which students in Sec B struggle with recursion?
    C->>L: question + tool schemas + user scope
    L->>C: call get_topic_weak_students(offering=B, topic=Recursion)
    C->>A: scoped call (same RBAC as UI)
    A->>C: 11 students, avg 38%, threshold 40%
    C->>L: tool result JSON
    L->>C: draft answer
    C->>C: verify every number appears in tool results
    C->>F: answer + tool calls + links
```

14.1 Components

- **Intent + slot resolution:** LLM function-calling picks a tool; names like "Section B" and "recursion" are resolved to IDs by a deterministic resolver against the user's accessible offerings/topics (not by the LLM guessing IDs).
- **Tool registry (~8 tools):** `get_offering_summary`, `get_topic_performance`, `get_weak_students(topic)`, `compare_assessments(a1, a2)`, `compare_sections`, `get_student_profile`, `get_attention_list`, `get_intervention_outcome`. Each wraps an existing analytics service — zero new maths.
- **Scope enforcement:** tools run as the logged-in user; a faculty cannot query another faculty's section through the copilot.
- **Number verifier:** regex-extract numbers from the answer; every number must match a value in tool outputs ($\pm$ rounding). Fail $\rightarrow$ regenerate once, then return the tool data as a table with no narrative.
- **Refusal path:** questions outside tools ("predict who will fail", "rank faculty") $\rightarrow$ polite fixed response listing what it can answer.
- **"Why did Section B decline?"** is answered as *where* it declined (topic-wise deltas CT1 $\rightarrow$ CT2, largest drops) — the system states correlation, not cause.
- **Privacy:** send register numbers + first names only when needed; log question, tools called, and latency in `ai_query_logs`, not full student data.
- **Provider:** any tool-calling LLM API; keep the client behind an interface so the model can be swapped. Budget ~\$5 of API credit for dev + demo.

14.2 Explicitly not doing

- Text-to-SQL (injection, wrong joins, scope leaks).
- RAG over PDFs of marks (numbers get paraphrased wrongly).
- Letting the LLM compute averages from lists.

15. UI/UX Structure

Navigation is organised around the faculty's question flow, not around database tables. A global **offering switcher** in the top bar (e.g. "DSA · CSE-B · Odd 2026") sets context for every page.

15.1 Navigation (faculty)

```
flowchart LR
    D[Dashboard] --- AS[Assessments]
    D --- AN[Analytics]
    D --- AT[Attention]
    D --- IN[Interventions]
    D --- ST[Students]
    D --- RP[Reports]
    D --- AI[Copilot V3]
    AN --- AN1[Topics]
    AN --- AN2[Questions]
    AN --- AN3[Sections]
    AS --- AS1[Marks grid]
    AS --- AS2[Import wizard]
```

Admin additionally sees **Administration** (users, departments, terms, courses, sections, offerings, thresholds, grading). "Marks" and "Topics" are not top-level: marks live inside an assessment; topics are configured per course and analysed under Analytics.

15.2 Screens

Screen	Answers the question	Key elements
Login	—	Email, password, error states
Dashboard	How is this offering doing right now?	KPI cards (students, mean, median, pass %, attention count); line chart: section mean per assessment; bar: 5 weakest topics; attention list preview; "latest import" card

Screen	Answers the question	Key elements
Assessments list	What have I assessed?	Table: name, type, date, max, weightage, completion %, mean; "New assessment"
Assessment detail	How did this test go?	Stats header; histogram; grade distribution; question table with difficulty chips; tabs: Overview / Marks / Questions / Import
Question & topic setup	What does each question test?	Editable question rows; multi-select topics with weight; live check "sum = max"
Marks grid	Enter/correct marks	Spreadsheet-like grid (keyboard nav), status toggle per cell, dirty-cell highlighting, save bar
Import wizard	Get marks in safely	4 steps: Upload → Map columns → Review (counters + error table, fix/exclude) → Done (summary + "view analytics")
Topic analytics	Where are they weak?	Horizontal bar per topic (avg %) with threshold line; topic × assessment heatmap; click topic → students below threshold
Question analytics	Which questions were hard or flawed?	Table: avg %, difficulty, (V2) discrimination, topic; flagged "review" questions
Section comparison	How do my sections compare?	Grouped bars: mean per section per assessment; topic table per section
Students list	Who is in this section?	Search, filters, trend chip, flag icon
Student profile	How is this student doing and why?	Header; assessment history line vs section mean; topic bars (strong/weak colours); trend with rule tooltip; flags with factors; generated insight sentence; interventions they were part of

Screen	Answers the question	Key elements
Attention	Who needs support?	Table sorted by severity; factor chips ("↓ Overall 44%", "↓ Recursion 31%"); bulk select → "Create intervention"
Interventions list	What have I tried?	Cards: topic, n students, date, status, net change badge
Intervention detail	Did it work?	Before/after dumbbell chart for target vs peers; net change; per-student table; caveat text
Reports	What do I hand the HOD?	Pick type + scope + format → download; preview for PDF
Copilot (V3)	Ask in plain English	Chat panel; each answer shows "Data used" expandable tool calls
Settings	How are labels computed?	Thresholds, grading bands, pass %; profile, password

15.3 Chart rules

- Each chart has a title phrased as the question it answers.
- Threshold lines drawn on topic and student charts so "weak" is visible, not just stated.
- No pie charts; no 3D; max 3 charts above the fold on the dashboard.
- Colour is never the only signal (add labels/icons) for accessibility.

16. Project Roadmap (Phases 0–12)

Phases overlap; the week column maps them onto the 8-week plan in Section 20. Phases 3–7 run in parallel across members.

Phase	Weeks	Goal	Key tasks	Depends on	Output	Definition of Done
0 Planning	0	Agree scope	Review this blueprint with guide; freeze MVP list; assign owners	—	Signed-off blueprint	Guide approves MVP; each member knows their first 3 tasks

Phase	Weeks	Goal	Key tasks	Depends on	Output	Definition of Done
1 Architecture	1	Contracts before code	Repo skeleton, folder ownership, OpenAPI stubs for all MVP endpoints, UI wireframes	0	Monorepo + <code>openapi.json</code> v0.1 + Figma/Excalidraw wireframes	Every MVP endpoint has request/response schema merged in <code>develop</code>
2 Database	1-2	Stable schema	SQLAlchemy models, Alembic migrations, seed script (realistic fake data: 3 sections × 65 students × 3 assessments)	1	<code>alembic upgrade head + make seed</code>	Fresh DB builds and seeds in one command on all 4 laptops
3 Backend	2-5	Core APIs	Auth/RBAC, org, students, assessments, marks; tests	2	Working endpoints	Endpoints pass pytest + Postman collection; RBAC tests green
4 Frontend	1-6	Usable app	Shell, auth, CRUD screens, dashboard, analytics pages	1 (mocks), 3 (real)	All MVP screens on real API	No mock data left; works at 1280 px
5 Analytics	2-5	Correct insights	Stats functions, summaries, topic, trend, flags, intervention outcome	2 (seed data)	analytics package + endpoints	Values match hand-computed Excel fixture to 2 dp
6 Excel import	2-4	Safe ingestion	Template, parser, validator, staging, confirm	3 (marks), 2	Import wizard end-to-end	All 10 validation rules covered by test files

Phase	Weeks	Goal	Key tasks	Depends on	Output	Definition of Done
7 Reports	5–6	Shareable output	Jinja PDF templates, XLSX/CSV exports	5	Section + assessment reports	3 formats download correctly; PDF reviewed by one faculty
8 AI	7	Stretch	Tool registry, LLM client, verifier, chat UI	5 complete	Copilot answering 5 demo questions	Verifier blocks a planted wrong number; scope test passes
9 Testing	3–8	Confidence	Unit, API, integration, E2E, security	all	CI green, test report	Coverage $\geq 70\%$ on analytics/import; 0 open P0 bugs
10 Deployment	6–8	Live URL	Dockerfiles, compose, VM, HTTPS, backups	3,4	Public demo URL	Fresh deploy from <code>main</code> via one command; HTTPS works
11 Documentation	6–8	Submission-ready	Report, API docs, user guide, README, architecture diagrams	all	Final report + README	Guide can run the app from README alone
12 Final demo	8	Win the room	Demo dataset, script, rehearsal $\times 3$, backup video	all	8-min demo + video	Rehearsed under 9 min twice with no failures

17. Four-Member Work Division

Split by vertical domain, not by layer. Each member owns the backend service, API and frontend screens of their domain, so a feature is finished by one person instead of waiting on three. Two horizontal responsibilities (database/platform, UI shell/design system) are assigned to one owner each so the codebase stays consistent.

flowchart LR

```

M1[M1 Platform &<br/>Foundation] --> M2[M2 Assessments<br/>& Data Ingestion]
M2 --> M3[M3 Analytics<br/>& Insights]
M3 --> M4[M4 Action Layer<br/>Interventions, Reports, AI]
M4 -.UI shell used by all.-> M1

```

Data flows left to right; the only hard handoffs are the schema (M1, Week 1) and the UI shell (M4, Week 1). Seed data lets M3 and M4 work before imports exist.

Workload balance (estimated)

Member	Backend	Frontend	Infra/Other	Est. effort
M1	55%	25%	20% (DB, CI, deploy)	~100 h
M2	55%	40%	5% (seed data)	~100 h
M3	60%	35%	5%	~105 h
M4	35%	50%	15% (AI, docs, demo)	~100 h

M1 — Platform & Foundation Lead

- **Role:** owns the skeleton everyone builds on: schema, auth, org structure, students, CI/CD, deployment.
- **Modules owned:** `core/`, `auth/`, `users/`, `org/`, `students/`, `alembic/`, `docker/`, `.github/workflows/`; frontend `pages/auth`, `pages/admin/*`, `pages/students/*`.
- **Technologies:** FastAPI, SQLAlchemy 2.0, Alembic, PostgreSQL, python-jose/PyJWT, passlib (argon2), pytest, Docker, GitHub Actions, nginx.
- **Build first:** repo skeleton + all SQLAlchemy models + first migration + auth (Week 1). Everyone is blocked on these; ship them rough and correct, polish later.
- **Deliverables:** migrations, auth with RBAC dependency (`require_role`, `require_offering_access`), org + student CRUD APIs and screens, student import (reusing M2's import engine), docker-compose, CI, deployed URL.
- **Dependencies:** none for Week 1. Uses M2's import engine for student import (Week 3). Uses M4's shell (Week 1-2).
- **Should NOT work on:** analytics maths, charts, import validation logic, AI.
- **Definition of Done:** fresh clone → `docker compose up` → seeded app with 3 roles logging in; every endpoint rejects unauthorised roles in tests; deployed on HTTPS.

M2 — Assessments & Data Ingestion

- **Role:** owns everything between "faculty has a paper script" and "clean marks are in the database".
- **Modules owned:** `assessments/`, `topics/`, `marks/`, `imports/`, `scripts/seed.py`; frontend `pages/assessments/*`, marks grid, import wizard, question/topic setup.

- **Technologies:** FastAPI, Pandas, OpenPyXL, RapidFuzz (name matching), React, a grid component (AG Grid Community or TanStack Table).
- **Build first:** the seed-data generator (Week 1) — realistic marks with built-in patterns (a weak Recursion topic, 5 declining students, one flawed question). This unblocks M3 and M4 immediately and becomes the demo dataset.
- **Deliverables:** assessment/question/topic CRUD + mapping UI, marks grid API + UI, generic import engine (used for marks and students), template generator, import wizard.
- **Dependencies:** M1 models + auth (Week 1). Calls M3's `recompute(assessment_id)` after writes — agreed as a stub function signature in Week 1.
- **Should NOT work on:** statistics, dashboards, reports, auth.
- **Definition of Done:** the 12 test files in `tests/fixtures/imports/` each produce the expected preview summary; confirm is atomic (killing the DB mid-import leaves no partial marks).

M3 — Analytics & Insights Engine

- **Role:** owns every number and label the system shows; the correctness owner.
- **Modules owned:** `analytics/` (pure functions + summary cache + attention rules + trend), `settings/` thresholds; frontend dashboard, topic analytics, question analytics, section comparison, student profile, attention list.
- **Technologies:** Python (statistics, NumPy/Pandas), SQL aggregates, pytest with hand-computed fixtures, Recharts.
- **Build first:** `analytics/stats.py` pure functions + a hand-computed Excel fixture that proves them (Week 2), then summary `recompute` over seed data.
- **Deliverables:** `recompute` pipeline, all `/analytics/*` endpoints, attention rules engine, trend classification, generated insight sentence, dashboard and analytics screens, (V2) discrimination + what-if.
- **Dependencies:** M1 schema; M2 seed data (Week 1) — no dependency on the import being finished.
- **Should NOT work on:** CRUD screens, import parsing, PDF layout, auth.
- **Definition of Done:** every metric in Section 12 has a unit test against the fixture; dashboard loads in < 300 ms p95 on seed data; every label has an explanation tooltip.

M4 — Frontend Platform & Action Layer

- **Role:** owns the app's look and feel and the "action + output" end of the loop: UI shell, interventions, reports, AI copilot, demo.

- **Modules owned:** frontend `app/`, `components/ui/*`, `lib/api/*` (generated client), routing, layout; backend `interventions/`, `reports/`, `copilot/`; `docs/`.
- **Technologies:** React, TypeScript, Vite, Tailwind, TanStack Query, React Router, openapi-typescript, Recharts; Jinja2 + WeasyPrint, openpyxl; LLM API (V3).
- **Build first:** app shell + design system + login page + typed API client (Week 1). Every other member builds screens on these components.
- **Deliverables:** shared component library (Table, Card, KPI, Chart wrapper, Modal, Form fields, EmptyState, ErrorState), interventions module (backend + UI), reports (PDF/XLSX/CSV), copilot (V3), user guide, demo script.
- **Dependencies:** M1 auth endpoints (Week 1); M3 topic-score function for intervention outcome (Week 4) and analytics endpoints for reports (Week 5).
- **Should NOT work on:** schema changes (request via M1), analytics formulas (call M3's functions), import logic.
- **Definition of Done:** all screens use shared components (no one-off styled tables); interventions show net change on demo data; all 3 report formats download; copilot passes the scope and number-verification tests (if built).

18. Task Backlog per Member

Difficulty: S (≤ 4 h), M (1 day), L (2–3 days). P0 = MVP-critical, P1 = MVP-important, P2 = V2/V3. Copy these into GitHub Issues with the ID as prefix.

M1 — Platform & Foundation

ID	Task	Description	Pri	Diff	Depends on	Output
M1-PL-01	Repo skeleton	Monorepo <code>backend/</code> , <code>frontend/</code> , <code>docker/</code> ; pre-commit (ruff, black, eslint, prettier); <code>.env.example</code>	P0	S	—	Cloneable repo
M1-DB-02	All models + migration 001	SQLAlchemy models for all 22 tables; Alembic initial migration	P0	L	01	<code>alembic upgrade head</code> works
M1-PL-03	docker-compose dev	postgres + api + frontend dev containers; <code>make up</code> , <code>make seed</code>	P0	M	02	One-command dev env
M1-AU-04	Auth	Login, refresh rotation, logout, <code>/me</code> , argon2 hashing	P0	M	02	Auth endpoints + tests

ID	Task	Description	Pri	Diff	Depends on	Output
M1-AU-05	RBAC dependencies	<code>require_role</code> , <code>require_offering_access(offering_id)</code> used by all routers	P0	M	04	Reusable guards + tests
M1-OR-06	Org APIs	Departments, terms, courses, sections, offerings CRUD	P0	M	05	Endpoints
M1-ST-07	Students API	CRUD, soft-deactivate, search (trigram), filters, pagination	P0	M	05	Endpoints
M1-FE-08	Admin screens	Users, courses, sections, offerings (assign faculty) using M4 components	P0	L	M4-FE-02	Admin UI
M1-FE-09	Students screens	List + filters + create/edit modal	P0	M	07	Students UI
M1-PL-10	CI	GitHub Actions: lint, pytest with Postgres service, frontend build	P0	S	01	Green CI on PRs
M1-ST-11	Student import	Plug student schema into M2 import engine	P1	M	M2-IM-06	CSV/XLSX student import
M1-SE-12	Security hardening	Login rate limit, CORS, security headers, upload limits	P1	S	04	Checklist done
M1-DP-13	Production deploy	Prod compose, nginx, HTTPS (Let's Encrypt), nightly pg_dump	P1	M	03	Public URL
M1-AU-14	Audit log	Write audit rows on marks/student changes	P1	S	02	audit_logs populated
M1-ST-15	Student role	Student login, read-only own profile	P2	M	M3-FE-11	Student view
M1-AU-16	Audit viewer + import rollback	Admin page; rollback a committed batch	P2	M	14	V2 feature

M2 — Assessments & Data Ingestion

ID	Task	Description	Pri	Diff	Depends on	Output
M2-SD-01	Seed data generator	Faker + patterned marks: 3 sections × 65 students, CT1/CT2/FT1, 13 DSA topics, weak Recursion, 5 decliners, 1 flawed question	P0	M	M1-DB-02	<code>make seed</code>

ID	Task	Description	Pri	Diff	Depends on	Output
M2-AS-02	Assessment API	CRUD on offering; weightage > 100 warning	P0	M	M1-AU-05	Endpoints
M2-TP-03	Topics API	Course topic tree CRUD (parent_id)	P0	S	M1-AU-05	Endpoints
M2-QN-04	Questions + mapping API	PUT /assessments/{id}/questions with topics + weights; sum = max check	P0	M	02, 03	Endpoint + tests
M2-MK-05	Marks grid API	GET grid; PUT changed cells; statuses; calls <code>recompute()</code> stub	P0	M	04	Endpoints
M2-IM-06	Import engine core	Generic parse → map → validate → stage → confirm, schema-driven	P0	L	05	<code>imports/engine.py</code>
M2-IM-07	Marks validators V1-V10	Implement rules + 12 fixture files	P0	L	06	Tests green
M2-IM-08	Template generator	XLSX with headers, max row, data validation	P0	S	04	Download endpoint
M2-FE-09	Assessment screens	List, create/edit, detail tabs	P0	M	M4-FE-02	UI
M2-FE-10	Question/topic setup UI	Rows, topic multi-select, weight, live sum	P0	M	04	UI
M2-FE-11	Marks grid UI	Keyboard nav, status toggle, dirty highlight, save	P0	L	05	UI
M2-FE-12	Import wizard UI	Upload → map → review (fix/exclude) → done	P0	L	07	UI
M2-IM-13	Name fuzzy match	RapidFuzz warning V8	P1	S	07	Warning shown
M2-IM-14	Import history	List batches per assessment	P1	S	06	UI list
M2-TP-15	Topic CSV import	Bulk topics per course	P2	S	03	Feature
M2-IM-16	Multi-sheet workbook	One sheet per section	P2	M	07	Feature

M3 — Analytics & Insights

ID	Task	Description	Pri	Diff	Depends on	Output
M3-AN-01	Hand-computed fixture	Excel workbook with 20 students × 2 assessments, every metric computed manually	P0	M	—	tests/fixtures/analytics.xlsx
M3-AN-02	Stats functions	mean, median, std, pass %, grades, histogram, completion — pure functions	P0	M	01	analytics/stats.py + tests
M3-AN-03	Recompute pipeline	recompute(assessment_id) → summary + topic summary rows	P0	L	M1-DB-02, M2-SD-01	Cache tables filled
M3-AN-04	Assessment + question endpoints	Stats, distribution, difficulty	P0	M	03	Endpoints
M3-AN-05	Topic analytics endpoint	Per assessment + cumulative, weak counts, drill-down list	P0	M	03	Endpoint
M3-AN-06	Trend + decline	Slope rule, labels, explanation text	P0	M	03	analytics/trend.py
M3-AN-07	Attention rules engine	R1-R6 from settings; flags with factors	P0	L	06	Flags table + endpoint
M3-AN-08	Dashboard endpoint	KPIs, series, weak topics, attention count	P0	M	04, 05, 07	Endpoint
M3-FE-09	Dashboard UI	KPI cards, trend line, weak-topic bars, attention preview	P0	L	M4-FE-02	Screen
M3-FE-10	Topic + question analytics UI	Bars with threshold, heatmap, difficulty table	P0	L	04, 05	Screens
M3-FE-11	Student profile UI	History vs mean, topic bars, trend, flags, insight	P0	L	06, 07	Screen
M3-FE-12	Attention list UI	Sorted table, factor chips, bulk select → intervention	P1	M	07	Screen
M3-AN-13	Section comparison	Endpoint + grouped bars	P1	M	04	Screen
M3-AN-14	Settings thresholds	Dept defaults + offering override API/UI	P1	M	07	Settings screen
M3-AN-15	Discrimination index	Upper/lower 27%	P2	S	04	Column in question table
M3-AN-16	What-if simulator	Endpoint + slider UI	P2	M	05	Feature
M3-AN-17	Z-score trends	Difficulty-adjusted trend option	P2	M	06	Toggle

M4 — Frontend Platform & Action Layer

ID	Task	Description	Pri	Diff	Depends on	Output
M4-FE-01	Vite + Tailwind + router	Project setup, env config, route guards by role	P0	S	M1-PL-01	Running SPA
M4-FE-02	Design system + shell	Sidebar, top bar with offering switcher, Table, Card, KPI, Modal, form fields, toasts, Empty/Error/Loading states	P0	L	01	<code>components/ui/*</code>
M4-FE-03	Typed API client	openapi-typescript generation script, fetch wrapper with token refresh, TanStack Query setup	P0	M	M1-AU-04	<code>lib/api</code>
M4-FE-04	Login + session	Login page, logout, 401 handling	P0	S	03	Auth flow
M4-FE-05	Chart wrappers	Line, bar, grouped bar, heatmap, dumbbell wrappers on Recharts with consistent styling	P0	M	02	<code>components/charts/*</code>
M4-IV-06	Interventions API	CRUD, target students, baseline/follow-up	P0	M	M1-AU-05	Endpoints
M4-IV-07	Outcome computation	Uses M3 topic-score function; target vs peers; net	P0	M	M3-AN-03	<code>interventions/outcome.py</code> + tests
M4-FE-08	Interventions UI	List, create (from attention selection), detail with dumbbell chart	P0	L	06, 07	Screens
M4-RP-09	Report engine	Jinja2 + WeasyPrint PDF base template, XLSX/CSV writers	P0	M	M3-AN-04	<code>reports/</code>
M4-RP-10	Section + assessment reports	Content, charts as static SVG/PNG, 3 formats	P0	L	09	Downloads
M4-FE-11	Reports UI	Pick type/scope/format, preview	P1	S	10	Screen

ID	Task	Description	Pri	Diff	Depends on	Output
M4-DC-12	User guide + README	Screenshots, setup, roles	P1	M	Week 6	docs/
M4-RP-13	Topic + department reports	V2 report types	P2	M	10	Downloads
M4-AI-14	Copilot backend	Tool registry, resolver, LLM client, verifier, logs	P2	L	M3 endpoints	copilot/
M4-AI-15	Copilot UI	Chat panel with "Data used" disclosure	P2	M	14	Screen
M4-DM-16	Demo package	Demo dataset script, rehearsal script, backup recording	P1	M	Week 7	Demo kit

19. Parallel Development Plan

Four people stay unblocked through three frozen contracts (schema, OpenAPI, component library), seed data, and folder ownership.

19.1 Contracts frozen in Week 1

- **Database contract:** M1 merges all models by end of Week 1. After that, schema changes go through a PR labelled `schema` that M1 reviews; one migration per PR; never edit a merged migration.
- **API contract:** every MVP endpoint exists as a stub returning example data by end of Week 1 (FastAPI + Pydantic models). Frontend generates types from `/openapi.json`. Changing a response shape = PR labelled `api-change` + ping the consumer.
- **Internal function contracts:** agreed signatures, stubbed Week 1:
 - `analytics.recompute(assessment_id) -> None` (M3 owns; M2 calls)
 - `analytics.topic_scores(student_ids, topic_id, assessment_ids) -> dict` (M3 owns; M4 calls)
 - `imports.engine.run(schema, file) -> Preview` (M2 owns; M1 calls for students)
- **UI contract:** M4's component library; others never create their own Table/Card/Modal.
- **Mocking rule:** frontend may use MSW mocks until the real endpoint merges; mocks are deleted in Week 4 (Integration 1). No mock data after that.

19.2 Folder ownership (CODEOWNERS)

```
backend/app/core/                @M1
backend/app/{auth,users,org,students}/ @M1
backend/alembic/                 @M1
backend/app/{assessments,topics,marks,imports}/ @M2
backend/scripts/seed.py         @M2
backend/app/{analytics,settings}/ @M3
backend/app/{interventions,reports,copilot}/ @M4
frontend/src/{app,components,lib}/ @M4
frontend/src/pages/{auth,admin,students}/ @M1
frontend/src/pages/{assessments,marks,imports}/ @M2
frontend/src/pages/{dashboard,analytics,attention}/ @M3
frontend/src/pages/{interventions,reports,copilot}/ @M4
docker/ .github/                 @M1
```

19.3 Git workflow

```
flowchart LR
    F1[feature/m1-auth] --> DEV[develop]
    F2[feature/m2-import-engine] --> DEV
    F3[feature/m3-topic-analytics] --> DEV
    F4[feature/m4-interventions] --> DEV
    DEV -->|end of each week, tagged| MAIN[main]
    H[hotfix/*] --> MAIN
```

- `main` : always deployable; updated only at weekly release (tag `v0.W`) and hotfixes.
- `develop` : integration branch; CI must be green.
- Branch naming: `feature/m{n}-{short-desc}`, `fix/m{n}-{desc}`, `chore/{desc}`.
- Branch lifetime ≤ 3 days. Rebase on `develop` daily. Big features land behind small PRs.

19.4 Commit and PR conventions

- Conventional Commits with task ID: `feat(imports): add V5 max-marks rule [M2-IM-07]`; types: feat, fix, refactor, test, docs, chore.
- PR template: what, why, task ID, screenshots (UI), test evidence, contract changes (yes/no).
- Every PR: 1 approving review (CODEOWNER of touched folders), CI green, no merge conflicts, $\leq \sim 400$ lines diff.
- Squash merge into `develop`; merge commit from `develop` $\rightarrow$ `main`.

19.5 Naming conventions

- Python: snake_case modules/functions, PascalCase classes, *_service.py , *_repository.py , schemas.py , router.py per module.
- DB: snake_case plural tables, {table}_id FKs, ix_{table}_{cols} indexes.
- API: plural nouns, kebab-free lowercase paths, JSON snake_case.
- TS: PascalCase components, useXxx hooks, one page folder per route.

19.6 Integration rhythm

- Daily 10-min stand-up (async in WhatsApp/Discord is fine): yesterday, today, blocked.
- **Integration Fridays:** merge develop → main , run full demo path on staging, file bugs.
- Two formal integration milestones: **I1 end of Week 4** (upload → dashboard end-to-end on real API), **I2 end of Week 6** (MVP feature freeze).
- Shared Postman/Bruno collection in repo; each backend owner adds their requests.

20. Week-by-Week Plan (8 weeks)

Integration happens at Week 4 and Week 6, leaving two full weeks for V2, hardening and demo prep. If the team only has 6 weeks, drop Week 7 (V2/V3) and compress Week 8 into Week 6.

Week	M1 Platform	M2 Ingestion	M3 Analytics	M4 Frontend & Action	Milestone at end of week
1	Repo, all models + migration, docker dev, auth, CI	Seed data generator; assessment/topic API stubs	Hand-computed fixture; stats.py; analytics endpoint stubs	Vite setup, shell, design system v1, login page, API client	Everyone can log in to the shell on seeded data; OpenAPI v0.1 frozen
2	RBAC guards; org + students APIs	Assessments, topics, questions + mapping APIs; assessment screens	Recompute pipeline over seed data; assessment + question endpoints	Chart wrappers; interventions API skeleton; offering switcher	CRUD for org, students, assessments working via API
3	Admin + students screens; audit log writes	Marks grid API + UI; import engine core; template	Topic analytics; trend + decline; dashboard endpoint	Dashboard UI with M3 (pair); intervention create UI	Faculty can enter marks manually and see stats

Week	M1 Platform	M2 Ingestion	M3 Analytics	M4 Frontend & Action	Milestone at end of week
4	Student import via engine; security hardening	Validators V1-V10 + fixtures; import wizard UI	Attention rules engine; dashboard UI finished; topic UI	Outcome computation; remove all mocks	I1: upload CT1.xlsx → validate → confirm → dashboard updates, on real API
5	Staging deploy (VM + HTTPS); backups	Fuzzy name match; import history; bug fixing	Student profile; attention list UI; question analytics UI	Interventions detail + dumbbell chart; report engine	Full loop works: flag → intervention → outcome
6	Performance check; deploy prod; bug fixing	Edge cases; UX polish on grid & wizard	Section comparison; settings thresholds; bug fixing	Section + assessment reports (PDF/XLSX/CSV); reports UI	I2: MVP feature freeze, all P0 done, CI green, deployed
7	Student role (P2) or test support	Import tests on real faculty sheet(s)	Discrimination index; what-if	Copilot backend + UI (P2)	V2 features merged; test report drafted
8	Final deploy, runbook	Demo dataset finalised with M4	Analytics correctness re-verification	User guide, demo script, backup video	Demo rehearsed 3×; report + README submitted

Buffer rule: if any P0 is late at the Week 4 checkpoint, Week 7 becomes P0 catch-up and V2/V3 is dropped. No exceptions.

21. MVP Definition

The project is successful when a faculty member can run the full loop below on a deployed URL, on real (or realistic) data, with no mock data and no manual database edits.

- ☐ **Authentication:** admin and faculty log in; faculty cannot see offerings they don't teach (tested).
- ☐ **Student management:** CRUD, search, filter, deactivate, bulk import.
- ☐ **Course/section management:** departments, terms, courses, sections, offerings with faculty assignment.
- ☐ **Assessment management:** assessments with questions and topic mapping.
- ☐ **Marks management:** grid entry/edit with absent/exempt status.

- ☐ **Excel import:** template → upload → preview with all 10 rules → fix/exclude → atomic confirm.
- ☐ **Core analytics:** all metrics in Section 12.3 correct against the fixture.
- ☐ **Dashboard:** KPIs, assessment trend, weak topics, attention preview per offering.
- ☐ **Topic analytics:** per-topic averages, below-threshold counts, drill-down to students.
- ☐ **Student profile:** history, topic profile, trend with rule, flags with factors.
- ☐ **Attention flags:** rules R1–R6 with configurable thresholds.
- ☐ **Interventions:** create from flagged students, outcome with peer comparison.
- ☐ **Reports:** section + assessment reports in PDF, XLSX, CSV.
- ☐ **Deployment:** Dockerised, HTTPS, reproducible from README.

Not required for MVP: AI copilot, student login, discrimination index, what-if, attendance, subtopics, department report.

Interventions are included in MVP deliberately: without them the product is a dashboard, and the "Action → Measurement" half of the vision disappears.

22. Wow Features (ranked for the demo)

Scored 1–5 on usefulness (U), technical depth (T), demo impact (D), feasibility (F). Build and rehearse in this order.

Rank	Feature	U	T	D	F	Why it earns its place
1	Smart import with live error preview and inline fix	5	4	5	4	Every faculty has lost time to bad spreadsheets; seeing "1 mark exceeds max" caught instantly lands immediately
2	Question → topic weak-area detection with drill-down	5	4	5	4	Turns marks into a teaching decision ("Recursion, 23 students")
3	Intervention outcome vs peer baseline	5	4	5	3	Closes the loop; the peer comparison shows statistical maturity examiners notice
4	Explainable attention flags with factor chips	5	3	4	5	Ethical, transparent alternative to "AI predicts failure"
5	Question quality: difficulty + discrimination, "review this question"	4	4	4	5	Helps faculty improve their papers, not just grade students; rare in student projects
6	One-click HOD-ready PDF report	4	3	4	4	Tangible artifact the guide can keep

Rank	Feature	U	T	D	F	Why it earns its place
7	Grounded copilot with "Data used" disclosure and number verifier	3	5	5	2	High impact but riskiest; demo only if it passed tests in Week 7
8	What-if simulator	3	2	4	5	Cheap; strong "aha" moment for HOD audiences

Not wow, skip: dark mode, animated charts, chat with emojis, gamified student badges, "AI risk score".

23. Project Novelty

ACADLYTICS's novelty is the closed, explainable loop from a single question's marks to a measured teaching action — not any single chart or algorithm.

Compared with	What they do	What ACADLYTICS adds
Excel	Flexible storage, manual formulas	Validated ingestion, zero re-work per assessment, access control, cross-assessment history, topic mapping
Basic student management systems	Student records, CRUD	Performance intelligence on top of records; hierarchy built for course offerings
Basic marks systems / ERP portals	Store totals, compute grades	Question-level data, topic mastery, trends, attention reasons, assessment quality
Simple dashboards	Charts of what happened	Why (topic/question decomposition), who (drill-down), what next (interventions), did it work (net change)

Strongest differentiators

1. **Question-to-topic weighted mapping** as a first-class data model, course-agnostic.
2. **Intervention measurement with a peer baseline** — honest about causality.
3. **Explainable-by-construction analytics** — every label carries its rule, threshold and n.
4. **Assessment quality analytics** (difficulty + discrimination) that improve the exam, not just grade it.

5. **Grounded AI pattern** (V3): LLM restricted to whitelisted tools, with automatic number verification.

24. Risks and Mitigations

The two risks most likely to sink this project are scope creep and late integration; both are addressed structurally, not by good intentions.

Category	Risk	Likelihood	Impact	Mitigation
Scope creep	Team starts AI/what-if before MVP works	High	High	MVP checklist gate; Buffer rule (Section 20); P2 issues locked until Week 7
Integration	Frontend built on mocks diverges from real API	High	High	OpenAPI-generated types; mocks deleted at I1 (Week 4); Integration Fridays
Technical	Real faculty sheets have merged cells, multiple header rows, per-section sheets	High	Medium	Template-first workflow; header detection in first 5 rows; column-mapping step; collect 3 real anonymised sheets in Week 1
Technical	Analytics numbers subtly wrong (absent as 0, mean of means)	Medium	High	Hand-computed fixture; M3 owns correctness; explicit missing-data rules (12.1)
Technical	Questions without topic mapping → empty topic analytics	Medium	Medium	Mapping UI in assessment creation; "x% of marks unmapped" warning on dashboard
Database	Schema churn breaks teammates	Medium	High	M1 sole migration owner; schema PR label; never edit merged migrations
Database	Data loss in demo environment	Low	High	Nightly pg_dump; seed script can rebuild demo in < 1 min
Security	Faculty accessing other sections via crafted API calls (IDOR)	Medium	High	require_offering_access on every scoped route; dedicated IDOR tests

Category	Risk	Likelihood	Impact	Mitigation
Security	Weak JWT handling / token leakage	Medium	Medium	Short-lived access tokens, rotating refresh, secret in env, HTTPS only
Security	Malicious upload (macro, zip bomb, huge file)	Low	Medium	5 MB cap, openpyxl read-only mode, no macro execution, extension + MIME check
AI	LLM invents or miscopies statistics	High	High	Tools-only access; number verifier; fall back to table; "Data used" disclosure; AI outside MVP
AI	Prompt injection via student names or free text	Low	Medium	Tool outputs passed as JSON data; system prompt forbids following instructions in data; no write tools
Privacy	Real student data in repo, logs or LLM prompts	Medium	High	Only synthetic data in repo; real data only on deployed DB with consent from guide; pseudonymise before LLM; logs exclude PII
Privacy	Stigmatising labels	Medium	Medium	Neutral wording ("additional academic support"), no ranks shown to students, flags visible only to faculty
Deployment	"Works on my machine"	Medium	Medium	Docker from Week 1; CI builds images; staging by Week 5
Deployment	WeasyPrint system dependencies fail in container	Medium	Low	Use official WeasyPrint-compatible base image; test PDF in CI
Team	One member drops out or falls behind	Medium	High	Vertical ownership keeps modules separable; each module has a named backup reviewer; weekly progress on board

Category	Risk	Likelihood	Impact	Mitigation
Validity	Intervention "improvement" challenged as regression to mean or easier test	High	Medium	Peer baseline and net change; explicit "observational" caveat

25. Testing Strategy

Testing effort is concentrated where wrong answers would be invisible: analytics, import, and authorization.

Layer	Tools	Scope	Target
Unit	pytest	Analytics functions, validators, trend, rules, outcome	≥ 90% on analytics/ and imports/validators
API	pytest + httpx TestClient, Postgres test DB	Every endpoint: happy path, validation error, 401, 403	All MVP endpoints
Integration	pytest	Import → recompute → dashboard values; intervention outcome from marks	5 end-to-end scenarios
UI	Vitest + React Testing Library	Shared components, import wizard steps, marks grid keyboard behaviour	Critical components
E2E	Playwright	Demo path (Section 26) scripted	Runs in CI nightly
Security	pytest + OWASP ZAP baseline scan	IDOR, auth, upload, headers	0 high findings
Performance	Locust (light)	Dashboard + import under 10 concurrent faculty	p95 < 300 ms reads

Key test cases

ID	Area	Case	Expected
T-AN-01	Analytics	Fixture: 20 students, known mean 61.25, median 60.00	Exact match to 2 dp
T-AN-02	Analytics	2 absent students	Excluded from mean; completion = 90%
T-AN-03	Analytics	Question mapped to 2 topics with weights 0.5/0.5	Each topic receives half the marks and half the max
T-AN-04	Analytics	Student scores 80, 70, 58	Slope -11 → Declining; explanation text contains "11"
T-AN-05	Analytics	Only 1 completed assessment	Trend = Insufficient data; no R4 flag
T-AN-06	Attention	Overall 44%, threshold 50	R1 fires, High severity, factor text correct
T-AN-07	Attention	Threshold changed from 50 to 40	Flag disappears after recompute
T-AN-08	Quality	Question where bottom group scores higher than top	Negative D, marked "review"
T-IV-01	Intervention	Target 42 → 67, peers 60 → 66	Net = +19 pp
T-IV-02	Intervention	Follow-up assessment has no question on topic	Outcome = "not measurable", not 0
T-IM-01	Import	Missing Register No column	File rejected with V1 message
T-IM-02	Import	Unknown register no	Row blocked, V2
T-IM-03	Import	Duplicate register no	Both rows blocked, V3
T-IM-04	Import	"twelve" in Q2	Cell blocked, V4
T-IM-05	Import	Q3 = 7 with max 5	Cell blocked, V5

ID	Area	Case	Expected
T-IM-06	Import	Blank cell / "AB"	Imported as absent
T-IM-07	Import	Header "q 1" instead of "Q1"	Auto-mapped
T-IM-08	Import	Confirm with blocking errors	409, nothing written
T-IM-09	Import	DB error mid-confirm	Transaction rolled back; 0 new marks
T-IM-10	Import	Re-import same assessment	Existing marks updated; audit rows contain old values
T-SE-01	Security	Faculty A requests Faculty B's offering analytics	403
T-SE-02	Security	Expired access token	401; client refreshes and retries
T-SE-03	Security	Reused refresh token	401 and token family revoked
T-SE-04	Security	50 MB upload / .exe renamed .xlsx	Rejected
T-SE-05	Security	SQL-ish input in search (' OR 1=1)	Treated as text; no error
T-AI-01	AI (V3)	Answer contains a number not in tool output	Verifier blocks; fallback table
T-AI-02	AI (V3)	Faculty asks about another faculty's section	Resolver returns no access; polite refusal

26. Final Demo Story (8 minutes)

The demo follows one faculty member, one section and one weak topic from upload to measured improvement. Demo data is seeded so CT1 and the remedial session already exist; CT2 is uploaded live.

Persona: Dr. Priya, DSA faculty, teaches CSE-A, B, C. Presenter speaks as her.

Time	Step	What's shown	Line to say
0:00	1. Hook	Title slide: a 186-row spreadsheet vs one dashboard	"Faculty spend hours computing averages and still can't answer: which topic, which students, did my help work?"
0:40	2. Login + dashboard	KPIs for DSA · CSE-B after CT1	"This is where CT1 left us: mean 58%, 12 students already flagged."
1:20	3. Upload CT2	Import wizard with a deliberately messy file	"Let me upload CT2 exactly as it came from evaluation."
2:00	4. Validation	Preview: 62 matched, 1 unknown reg no, 1 duplicate, Q4 = 7 of 5	Fix Q4 inline, exclude the duplicate, confirm: "Three errors that would have silently corrupted my Excel averages."
2:50	5. Dashboard updates	Trend line gains CT2 point; weak-topic bar shows Trees dropping	"Section mean fell 6 points. Why?"
3:20	6. Weak topic	Topic analytics: Trees 41%, 23 students below 40%	"The decline is concentrated in Trees — Q4 and Q5."
3:50	7. Question quality	Q5 flagged: negative discrimination	"Strong students did worse than weak ones on Q5 — the question itself may be ambiguous."
4:20	8. Student profile	Open a declining student: 78 → 66 → 51, factor chips	"Every flag shows its rule and threshold. No black box."
5:00	9. Create intervention	From attention list: select 17 students → "Trees remedial, 2 Oct"	"Insight becomes an action in two clicks."
5:40	10. Measure	Pre-seeded earlier intervention on Recursion (CT1 → CT2)	"Targeted students +25 points, others +6: a net +19 point difference. We call it observational, not proof."
6:30	11. Copilot (only if stable)	"Which students in Section B are struggling with recursion?" → answer + "Data used"	"The AI can't invent numbers — it only reads what the analytics engine computed, and we verify every figure."

Time	Step	What's shown	Line to say
7:10	12. Report	Download section PDF for the HOD	"And this is what goes to the HOD."
7:40	Close	Architecture slide + future scope	"Data, insight, action, measurement — in one loop."

Demo safety rules

- Run on deployed URL **and** a local docker instance; switch if the network fails.
- A pre-recorded video of the full path as last-resort backup.
- Reset script restores the pre-demo state in < 60 s between rehearsals.
- If copilot is not green by Week 7 testing, cut step 11; the demo still works end to end.

27. Final Project Report Summary

For sharing with the faculty guide.

Item	Summary
Project name	ACADLYTICS — Academic Performance Intelligence Platform
Problem	Faculty manage question-wise marks for multiple sections in disconnected spreadsheets; analysis of weak topics, student trends and the effect of remedial actions is manual, slow or missing
Solution	A web platform that validates and imports assessment data, computes explainable analytics down to question and topic level, flags students needing academic support with visible reasons, and measures the effect of faculty interventions against a peer baseline
Target users	HOD/Admin, Faculty; Students (read-only, optional)

Item	Summary
Core features	Role-based access; department/course/section/offering management; assessments with question-topic mapping; smart Excel import with preview and validation; statistics, topic and question analytics; trend classification; rule-based attention flags; interventions with outcome measurement; PDF/Excel/CSV reports
Technology stack	React, TypeScript, Tailwind, Recharts, TanStack Query; Python, FastAPI, Pydantic, SQLAlchemy, Alembic; PostgreSQL; Pandas, OpenPyXL; JWT + argon2; WeasyPrint; Docker, GitHub Actions; cloud VM (AWS/Azure)
Architecture	Modular monolith: SPA → REST API (auth, RBAC) → service layer → deterministic analytics engine → PostgreSQL, with an atomic import pipeline and write-time analytics caching
Innovation	Weighted question-to-topic model; closed Data → Insight → Action → Measurement loop; explainable- by-construction flags; assessment quality analysis; grounded AI design where the LLM cannot generate statistics
Expected outcome	Assessment processing reduced from hours to minutes; evidence-based identification of weak topics and students; measurable, documented interventions; reusable across all CSE courses
Future scope	Grounded AI copilot; attendance integration; concept-level analytics; ERP integration; multi- department deployment; difficulty-adjusted longitudinal tracking across semesters